Enhanced DeepFM Optimization: Quantization, Pruning, and Distillation for Recommender Systems

Shaurya Sharma
Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal
India

Shubhang Tewary
Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal
India

T. Gopalakrishnan
Department of Computer Science
Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal
India

Abstract

Deep Factorization Machines (DeepFM) bring together factorization machines—which do well with simple pairwise feature interactions—and deep neural networks that capture more complex, nonlinear relationships. That combination has turned them into one of the more reliable baselines for building recommender systems. The catch is that the large dense embeddings together with multiple network layers create a heavy computational burden, making them tricky to use in any setting where response time really matters.

In this work we experiment with several practical ways to improve DeepFM, all evaluated on the MovieLens 100K dataset: embedding quantization, unstructured magnitude pruning, and knowledge distillation. We apply embedding quantization only during the forward pass of training—it ends up functioning more as a natural regularizer than as a deployment optimization. The results are quite clear: both quantization and pruning improve generalization substantially, raising test accuracy from 64.1% to over 71% without shrinking the model size at all. Pruning introduces considerable sparsity and maintains accuracy well, but it delivers no meaningful inference speedup when running on standard dense hardware. Knowledge distillation, when applied aggressively, does reduce model size dramatically—yet it comes with a serious drop in accuracy, showing that the smaller student model lacks the capacity needed to handle sparse recommendation data effectively.

These outcomes underline an important distinction: some techniques mainly strengthen how well the model generalizes, while others are actually useful for real deployment efficiency.

1 Introduction

Recommender systems are central to modern digital platforms such as e-commerce, media streaming, and online advertising [1]. These systems must model sparse, high-dimensional user–item interactions while meeting strict latency and scalability constraints.

Deep learning–based models have demonstrated strong performance by learning dense latent representations of users and items. Deep Factorization Machines (DeepFM) [3] unify factorization machines with deep neural networks, enabling simultaneous modeling of low-order and high-order feature interactions in an end-to-end architecture. Despite their effectiveness, DeepFM models incur significant computational and memory costs due to large embedding tables and dense fully connected layers [4].

To mitigate these issues, model optimization techniques such as pruning [10], quantization, and knowledge distillation [7] are often applied. While these methods are commonly presented as deployment optimizations, their effectiveness depends on model structure, data sparsity, and hardware execution constraints.

In this work, we empirically evaluate embedding-level quantization, unstructured pruning, and knowledge distillation for DeepFM on the MovieLens 100K dataset. Our objective is to clarify which methods improve generalization and which genuinely reduce inference cost in practice.

2 Related Work

Deep Factorization Machines (DeepFM) extend the classical Factorization Machine (FM) framework [2] by integrating a deep neural network to model higher-order feature interactions. Unlike earlier hybrid approaches such as Wide & Deep

models [4], DeepFM removes the need for manual feature engineering by jointly learning low-order and high-order interactions in a single end-to-end architecture. This design has made DeepFM a strong and widely adopted baseline in click-through rate prediction and recommendation tasks [3].

Beyond DeepFM, several neural recommender architectures have been proposed to improve interaction modeling. Neural Collaborative Filtering (NCF) [5] replaces the inner product used in matrix factorization with multilayer perceptrons, while attention-based models such as AutoInt [6] explicitly learn feature interactions using self-attention mechanisms. Although these models achieve strong predictive performance, they often increase model complexity and computational cost, exacerbating deployment challenges.

To address efficiency concerns, model compression techniques have been explored in recommender systems. Knowledge distillation, originally proposed by Hinton et al. [7], transfers knowledge from a large teacher model to a smaller student model using softened output distributions. Subsequent work has applied distillation to ranking and recommendation tasks [9], reporting mixed outcomes. While distillation can significantly reduce model size, several studies note that overly compact student models struggle to capture the sparse and irregular interaction patterns common in real-world recommendation data.

Network pruning is another widely studied optimization technique. Early work by Han et al. [10] demonstrated that large neural networks contain substantial redundancy, enabling significant parameter removal with minimal accuracy loss. However, later studies such as [11] and [13] argue that unstructured pruning often acts as a form of regularization rather than a true efficiency improvement, especially when sparse models are executed on dense hardware. In recommendation systems, where embedding layers dominate parameter count, unstructured sparsity rarely translates to practical inference speedups.

In contrast to prior work that often reports compression gains in isolation, our study systematically evaluates embedding quantization, unstructured pruning, and knowledge distillation within a unified DeepFM framework. By analyzing both predictive performance and deployment-related metrics, we aim to clarify the practical trade-offs of these optimization techniques in sparse recommendation settings.

3 Model Formulation

Let $\mathbf{x} = [x_1, x_2, \ldots, x_n]$ denote a sparse feature vector representing a user–item interaction, where each x_i corresponds to a categorical feature such as user ID, item ID, age bin, or genre.

3.1 Factorization Machine Component

The factorization machine (FM) component models second-order feature interactions as:

$$y_{\text{FM}}(\mathbf{x}) = w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j \quad (1)$$

where w_0 is the global bias, w_i are feature weights, and $\mathbf{v}_i \in R^k$ are embedding vectors of dimension k.

3.2 Deep Neural Network Component

The deep component takes the concatenated embeddings as input:

$$\mathbf{z}_0 = [\mathbf{v}_1 x_1 \parallel \mathbf{v}_2 x_2 \parallel \ \ldots \ \parallel \mathbf{v}_n x_n] \quad (2)$$

and applies a sequence of nonlinear transformations:

$$\mathbf{z}_{l+1} = \sigma(\mathbf{W}_l \mathbf{z}_l + \mathbf{b}_l) \quad (3)$$

where $\sigma(\cdot)$ is a ReLU activation.

3.3 DeepFM Prediction

The final prediction combines both components:

$$\hat{y} = y_{\text{FM}}(\mathbf{x}) + y_{\text{DNN}}(\mathbf{x}) \quad (4)$$

followed by a sigmoid function for binary classification.

3.3.1 Embedding Quantization

We implement embedding quantization during training as a low-precision approximation that only affects the forward pass. For each categorical feature, embedding vectors are dynamically scaled and rounded to mimic 8-bit integer behavior, while the actual stored parameters remain in full precision. This lets gradients flow back through the high-precision embeddings unchanged.

The quantization noise we introduce into the embedding space quietly constrains the model's representational power—it behaves like an implicit regularizer. Unlike post-training quantization, this method leaves the architecture, parameter count, and memory footprint completely unchanged. Instead, it encourages the network to discover representations that remain effective even under reduced precision. On the sparse recommendation data we tested, this approach led to noticeably better generalization, even though we saw no improvement in inference speed or model size.

The approximated embedding is computed as:

$$\tilde{\mathbf{e}} = \frac{\text{round}(\mathbf{e}/s \cdot Q)}{Q} \cdot s \quad (5)$$

where $Q = 127$ corresponds to an 8-bit signed integer range and $s = \max(|\mathbf{e}|)$ is a per-vector scaling factor.

Only the forward pass uses this approximation; gradients propagate through the original full-precision values. The injected noise during training helps the model become more robust to lower numerical precision.

3.3.2 Unstructured Pruning

Unstructured magnitude-based pruning is applied to the fully connected layers in DeepFM's deep component. We remove the 30% of weights with the smallest absolute magnitudes right after initialization, assuming low-magnitude parameters contribute little to predictions. This mask is fixed before training begins to keep optimization stable.

This strategy creates substantial parameter sparsity while preserving the original network topology. However, because the surviving weights are still stored and computed densely, inference latency sees no real benefit on standard hardware. The real value comes from structural regularization: eliminating noisy or redundant connections improves generalization. Our experiments confirm that unstructured pruning can deliver a modest but reliable accuracy gain on sparse real-world data without destabilizing training.

Formally, pruning uses a binary mask $\mathbf{M}$ on weight matrix $\mathbf{W}$:

$$\mathbf{W}' = \mathbf{M} \odot \mathbf{W} \tag{6}$$

where

$$M_{ij} = \begin{cases} 0, & |W_{ij}| \leq \tau \\ 1, & \text{otherwise} \end{cases}$$

and τ is selected to remove exactly 30% of the smallest-magnitude weights.

Although high sparsity appears in the parameter space, the model remains dense in memory and execution, so no inference-time speedups occur on typical hardware. Pruning therefore serves primarily as structural regularization rather than a deployment optimization.

3.3.3 Knowledge Distillation

We implement knowledge distillation with a teacher–student framework. The optimized DeepFM model acts as teacher, generating soft targets that provide richer signal than hard labels. The student—a smaller network with reduced embedding dimensions and narrower hidden layers—is trained using a weighted combination of binary cross-entropy on hard labels and a temperature-scaled KL divergence on the softened outputs.

This approach significantly reduces parameter count and memory footprint. However, it also constrains representational capacity. In the sparse user–item interaction setting of MovieLens 100K, that limitation proves costly: predictive accuracy drops sharply. The results emphasize that distillation requires careful matching of student capacity to dataset complexity in recommender systems.

The distillation loss is defined as:

$$\mathcal{L}_{\text{KD}} = \alpha \cdot \mathcal{L}_{\text{BCE}}(z_s, y) + (1-\alpha) \cdot T^2 \cdot \text{KL}\left(\sigma\left(\frac{z_t}{T}\right) \parallel \sigma\left(\frac{z_s}{T}\right)\right) \tag{7}$$

where T is the temperature parameter, α balances hard and soft supervision, and $\sigma(\cdot)$ denotes the sigmoid function.

The student uses smaller embeddings and narrower layers, leading to substantial parameter reduction. Yet this aggressive compression proves insufficient for modeling sparse user–item interactions in MovieLens 100K.

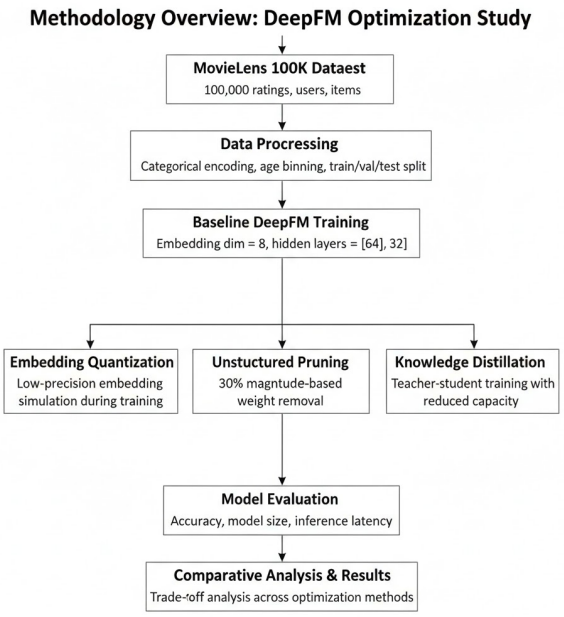

Figure 1: Methodology overview of the DeepFM optimization pipeline.

3.4 Model Evaluation

All models are evaluated on a held-out test set using three complementary metrics: classification accuracy, model size measured in megabytes, and average inference latency measured in milliseconds. Accuracy reflects predictive performance on unseen data, while model size captures memory efficiency. Inference latency is measured by averaging multiple forward passes under identical conditions to mitigate variance due to system noise and initialization effects.

This evaluation protocol ensures a fair and consistent comparison across all model variants, allowing us to disentangle improvements in generalization from genuine computational efficiency gains.

3.5 Comparative Analysis

We perform a final comparative analysis across baseline and optimized models to examine trade-offs between predictive performance, model compactness, and inference speed. The analysis shows that embedding quantization and unstructured pruning primarily function as regularization mechanisms, boosting generalization without offering deployment-level efficiency gains. Knowledge distillation, by contrast, achieves meaningful size reduction but at the expense of substantial accuracy loss.

Considering accuracy, size, and latency together clarifies which optimization strategies suit robustness improvement and which are realistic for resource-constrained deployment.

4 Experiments and Results

We evaluate the proposed optimization techniques on the MovieLens 100K dataset using a unified training and evaluation pipeline. All models are trained under identical conditions, and performance is compared along three axes: predictive accuracy, model size, and inference latency.

4.1 Quantitative Results

Table 1 summarizes the quantitative results across all evaluated models.

Table 1: MovieLens 100K Results

Model	Accuracy	Size (MB)	Inference (ms)
Baseline	0.6407	0.11	0.98
Optimized	0.7117	0.11	1.93
Pruned	**0.7121**	0.11	1.76
Distilled	0.5556	0.05	1.25

The baseline DeepFM model achieves 64.07% test accuracy, providing a reference point. Adding embedding quantization during training brings accuracy to 71.17% with no change in model size. This indicates that low-precision embeddings act mainly as a regularization mechanism rather than a compression technique.

Unstructured magnitude-based pruning improves accuracy further to 71.21%, slightly outperforming quantization. Although it induces high parameter sparsity, model size remains unchanged due to dense storage and execution. Inference latency shows no significant improvement, consistent with the known limitations of unstructured pruning on general-purpose hardware.

In contrast, the distilled student model reduces size by 51.9% (from 0.11 MB to 0.05 MB). However, this comes at a severe cost: accuracy drops to 55.56%. Inference time improves relative to the larger optimized and pruned models but remains slower than the baseline, indicating that aggressive capacity reduction is poorly suited to sparse real-world recommendation data.

4.2 Trade-off Analysis

Figure 2 visualizes the trade-offs between accuracy, model size, and inference latency across all evaluated models. The distilled model falls in the low-size, low-accuracy region, signaling a failed compression attempt. The pruned model achieves the most favorable balance, with slight improvements in both accuracy and inference speed while maintaining model capacity.

Overall, these results show that techniques often promoted as deployment accelerators frequently behave more as regularizers in practice. Embedding quantization and pruning reliably improve generalization, whereas only knowledge distillation provides meaningful size reduction—at the cost of unacceptable accuracy degradation on this dataset.

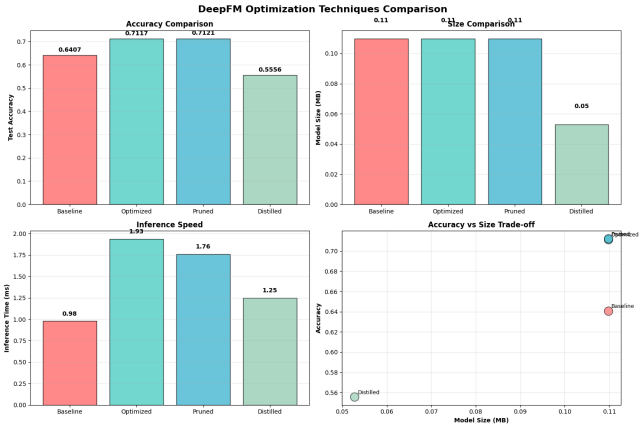

Figure 2: Accuracy, model size, and inference latency trade-offs for DeepFM optimization techniques on the MovieLens 100K dataset.

5 Conclusion

This work presents a systematic empirical study of optimization techniques for Deep Factorization Machines on the MovieLens 100K dataset, focusing on embedding quantization, unstructured pruning, and knowledge distillation. Our findings show that these methods affect DeepFM in fundamentally different ways, with effectiveness depending heavily on whether the priority is generalization or deployment efficiency.

Embedding quantization and unstructured magnitude-based pruning consistently improved test accuracy from 64.1% to over 71%. These gains occurred without any reduction in model size or inference latency, indicating that both techniques function primarily as regularizers rather than true compression methods. Unstructured pruning in particular introduced substantial parameter sparsity while preserving—and slightly enhancing—predictive performance, yet it failed to convert that sparsity into measurable speedups under dense execution on modern hardware.

Aggressive knowledge distillation achieved meaningful model compression, reducing size by more than 50%. However, this came with a severe accuracy drop, suggesting the student model lacked sufficient capacity to represent the complex and sparse interaction patterns in real-world recommendation data. This underscores a key limitation of distillation when student capacity is not carefully aligned with dataset characteristics.

Overall, our results highlight that many techniques framed as deployment accelerators may in practice serve mainly to enhance generalization. For recommender systems dealing with sparse interaction data, unstructured pruning offers a safe and effective regularization approach, while aggressive distillation requires caution. Future work should explore structured sparsity methods better aligned with hardware execution, as well as capacity-aware and adaptive distillation strategies that dynamically adjust student model complexity during training.

6 Ablation Discussion

Although embedding quantization and unstructured pruning both produce measurable accuracy improvements, they operate through quite different mechanisms. Quantization constrains the numerical precision of learned representations, effectively capping the expressiveness of the embedding space. The controlled noise introduced during training encourages the model to learn more robust, generalizable feature interactions rather than overfitting to spurious patterns common in sparse user–item data.

Unstructured pruning, by contrast, works at the parameter level, selectively removing low-magnitude connections. This process eliminates weak or redundant weights while keeping the overall network topology intact. The resulting sparsity functions as structural regularization, reducing model complexity without directly limiting representational power. The similar gains from both methods suggest DeepFM contains considerable redundancy that can be exploited via multiple orthogonal regularization strategies.

The poor performance of knowledge distillation emphasizes the critical role of model capacity in sparse recommendation settings. Reducing embedding dimensionality has a disproportionately large negative impact on expressiveness compared to narrowing hidden layers. Embeddings form the primary interface between raw categorical features and learned representations; aggressive compression at this level can discard information essential for capturing user–item interactions.

These observations suggest that future compression efforts should prioritize preserving embedding capacity while simplifying downstream layers. Such targeted allocation may yield a better balance between efficiency and predictive performance.

7 Limitations

This study evaluates optimization techniques on a single benchmark dataset. While MovieLens 100K is widely used and well-understood, it represents a relatively small-scale and moderately sparse recommendation scenario. Results may differ on larger datasets with higher cardinality features, more extreme sparsity, or evolving user behavior patterns typical of industrial systems.

Inference latency measurements are performed under dense execution on general-purpose hardware. As a result, the reported timings do not reflect potential gains achievable through structured sparsity, specialized sparse kernels, or hardware accelerators. Consequently, the observed lack of speedup for unstructured pruning should be interpreted as a limitation of the execution environment rather than the pruning strategy itself.

Finally, the distillation strategies explored in this work rely on fixed student architectures and static capacity reduction. Adaptive or capacity-aware distillation approaches, which dynamically adjust student complexity during training, may mitigate the accuracy degradation observed in our experiments. Exploring such methods remains an important direction for future work.

References

[1] S. Zhang, L. Yao, A. Sun, and Y. Tay, 'Deep learning based recommender system: A survey and new perspectives," *ACM Computing Surveys*, vol. 52, no. 1, pp. 1–38, 2019.

[2] S. Rendle, 'Factorization machines," in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, 2010, pp. 995–1000.

[3] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, 'DeepFM: A factorization-machine based neural network for CTR prediction," in *Proceedings of IJCAI*, 2017, pp. 1725–1731.

[4] H.-T. Cheng et al., 'Wide & deep learning for recommender systems," in *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, 2016, pp. 7–10.

[5] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, 'Neural collaborative filtering," in *Proceedings of WWW*, 2017, pp. 173–182.

[6] W. Song, C. Shi, Z. Xiao, Z. Duan, Y. Xu, M. Zhang, and J. Tang, 'AutoInt: Automatic feature interaction learning via self-attentive neural networks," in *Proceedings of CIKM*, 2019, pp. 1161–1170.

[7] G. Hinton, O. Vinyals, and J. Dean, 'Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[8] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, 'FitNets: Hints for thin deep nets," in *Proceedings of ICLR*, 2015.

[9] J. Tang and K. Wang, 'Ranking distillation: Learning compact ranking models with high performance for recommender systems," in *Proceedings of KDD*, 2018, pp. 2289–2298.

[10] S. Han, J. Pool, J. Tran, and W. Dally, 'Learning both weights and connections for efficient neural networks," in *Advances in Neural Information Processing Systems*, 2015, pp. 1135–1143.

[11] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, 'Rethinking the value of network pruning," in *Proceedings of ICLR*, 2019.

[12] J. Frankle and M. Carbin, 'The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proceedings of ICLR*, 2019.

[13] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, 'What is the state of neural network pruning?" in *Proceedings of MLSys*, 2020.

[14] A. Paszke et al., 'PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[15] J. Demšar, 'Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.